2025 K-Software Empowerment Bootcamp

Auto Battle Tactics

팀명: IDLE

2025.08.26

프로젝트 분야	□ 인공지능(AI) / □ 스마트팩토리 / ■ 게임
소스코드 URL	https://github.com/KEB-IDLE/Auto-Battle-Tactics
시연영상 URL	https://www.youtube.com/@dhwotlr/videos

목차

1. 프로젝트 개요 **3**

1.1 프로젝트 요약 3

1.2 프로젝트 목표 및 내용 3

1.3 프로젝트 팀 구성 및 역할 4

2. 프로젝트 주차별 보고서 **4**

2.1 주차별 프로젝트 작업 현황 요약표 4

3. 프로젝트 결과 보고 **5**

3.1 개요 **5**

3.2 게임 기획 **7**

3.2.1 세계관 및 콘셉트 7

3.2.2 게임 시스템 설계 9

3.3 구현 완성도 **12**

3.3.1 핵심 기능 구현 12

3.3.2 게임 진행 흐름 15

3.3.3 기술 안정성 및 버그 최소화 17

3.3.4 게임 밸런스 18

3.4 아트·UX/UI 구성 **19**

3.4.1 유닛 19

3.4.2 맵 19

3.4.3 인게임 UI 21

3.4.4 AI 이미지 생성 과정 25

3.5 기술 및 구조 **31**

3.5.1 시스템 아키텍처 31

3.5.2 주요 모듈 및 서비스 흐름 32

3.5.3 기술 스택 및 도입 목적 33

3.5.4 애니메이션 적용 34

1. 프로젝트 개요

1.1 프로젝트 요약

구분	내용
프로젝트명	Auto Battle Tactics
수행기간	2025.07.01 ~ 2025.08.28 (총 8주)
주요목표	**싱글 플레이 자동 전투 전략 게임 개발**
활용기술	클라이언트 : C# 9.0, .NET Framework, Unity 6.19 서버 : JavaScript (ES2024), Node.js v22.17.0, Express, MySQL, Swagger 인프라 : AWS EC2, Docker, Jenkins, NGINX AI: AUTOMATIC1111 협업도구: GitHub, Discord, Notion
개발환경	Windows 11
적용 분야	싱글 플레이 전략 게임 개발 AI 기반 이미지 및 콘텐츠 제작 게임 서버 확장을통한 온라인 기능 및 부가 서비스 지원

1.2 프로젝트 목표 및 내용

구분	내용
프로젝트 목표	Auto Chess / TFT 스타일의 싱글 플레이형 전략 자동 전투 게임 개발
주요 내용	1. 개성있는 스토리와 차별화된 유닛들 2. 유닛 시너지 및 전략적 전투 요소 3. 다양한 테마의 맵과 전술 요소 4. 게임 서버 기반의 확장 기능 지원 5. LoRA 기반 AI 이미지 생성 모델을 이용한 게임 에셋 제작 및 적용

1.3 프로젝트 팀 구성 및 역할

이름	소속	역할	담당업무	참여기간
오재식	인하대	서버/인프라	백엔드 구현, 클라우드 배포, CI/CD 자동화	8주
안현수	경기대	기획/클라이언트	콘텐츠 기획, 유니티 개발, 보고서 작성	8주
엄지영	경기대	기획/문서	게임 기획, 자료 제작(설계문서·발표·영상)	8주
이성재	경기대	AI	데이터 수집, 모델링, AI 아트워크	8주
이승혁	경기대	클라이언트	유니티 개발, 맵 제작	8주

2. 프로젝트 주차별 보고서

2.1 주차별 프로젝트 작업 현황 요약표

주차	주요 활동	이슈 및 해결
1주차 (7/1~7/4)	시장조사, 게임 콘셉트 기획	계획서 완성
2주차 (7/7~7/11)	유닛 기획, 서버 설계	초기 게임 기획 및 기본 아키텍처 구축
3주차 (7/14~7/18)	메인 씬 제작, UI 프로토타입	핵심 기능 프로토타입 구현
4주차 (7/21~7/25)	AI 작업, 서버 클라우드 배포	클라우드 테스트 환경 확보
5주차 (7/28~8/1)	게임 씬 제작, UI 개선	시각적 완성도 및 몰입도 강화
6주차 (8/4~8/8)	맵 제작, 콘텐츠 확장	전체 게임 흐름 완성 AI 적용 마무리
7주차 (8/11~8/15)	테스트, 밸런싱	안정적 플레이 확보
8주차 (8/18~8/22)	보고서 작성, 발표자료	발표 준비 완료
9주차 (8/25~8/28)	피드백 반영, 제출물 보완	프로젝트 최종 마무리

3. 프로젝트 결과 보고

3.1 개요

오토배틀 장르 시장 동향과 대표작

기존 전략 게임의 긴 플레이 시간과 반복적 진행에 따른 피로

전략 자동 전투 게임 수요 증가

오토배틀 장르 시장 규모 2025년 기준 약 15억 달러 추정 및 연평균 15% 성장세

대표작으로 도타2의 오토체스와 리그오브레전드의 TFT.

게임 산업 속 AI 적용과 전망

AI 기반 게임 제작 매출 급성장

초보자와 숙련자 간 실력 격차로 인한 진입장벽

Pay-to-Win 구조로 인한 불공정성에 따른 사용자 불만

소규모 개발 환경에서 고품질 에셋 제작에 필요한 시간과 비용 부담

게임 완성도 및 기존 유저 장기적 유지에 부정적 영향

타겟 시장 및 분석

반복 라운드 플레이로 전략적 재미를 경험하고 싶은 오토배틀러 유저

조작 부담 없이 전략에 집중하고 싶은 싱글 플레이 전략 게임 유저

캐릭터 수집 요소에 관심 있는 전략 게임 관심 유저

타겟 유저 핵심 요구 사항

간단한 조작과 직관적인 배치 시스템

캐릭터 수집 요소 관심

매력적인 세계관과 개성 있는 캐릭터 중요시

짧은 시간으로 전략적 재미 경험

수익구조

기본 수익 모델

Freemium (기본 무료 + 선택적 과금) 방식을 채택하여, 게임플레이에 있어 진입장벽을 낮춤

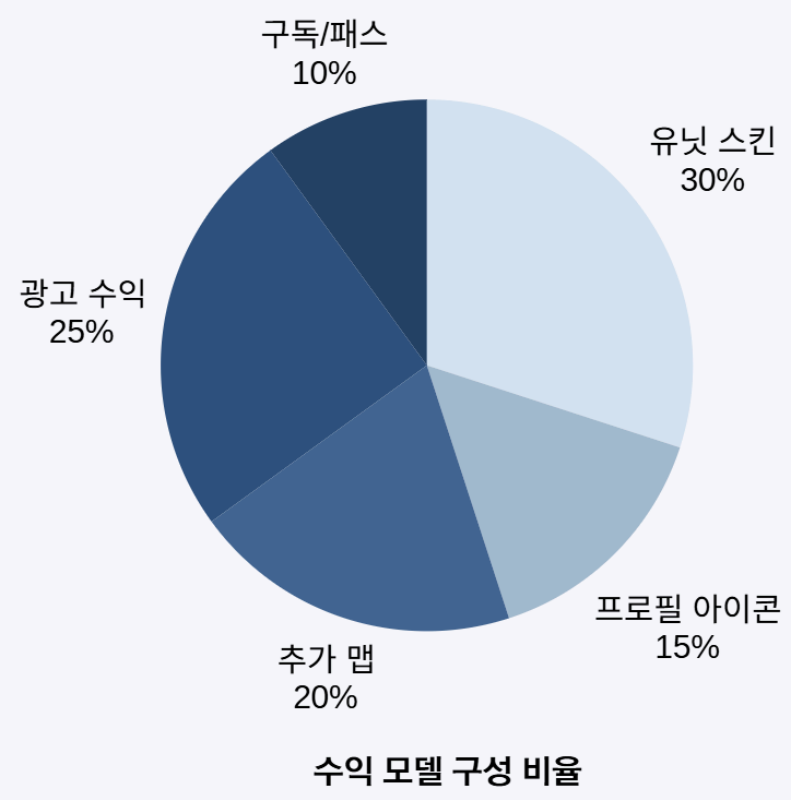
주요 BM 요소

유닛 코스튬/스킨 판매 (비전투성, P2W 방지)

확장 콘텐츠 DLC (유닛/맵/챕터)

수익 구조 방향성

수집 / 꾸미기 중심으로, 실제 게임 플레이에는 영향을 주지 않는 방향의 수익구조 설계



목표 및 차별점

간단한 조작 + 자동 전투 시스템으로 신규·라이트 유저도 접근 가능

실력과 운의 조합으로 초보자와 숙련자 모두 만족 가능

커스텀 해금 시스템 → 성취 목표와 장기적 플레이 동기 제공

게임 서버를 통한 이벤트/신규 콘텐츠 확장 기반 마련 → 유저 유지율 확보

구분	차별점
싱글 플레이	PvP 피로감 최소화
비용 절감	독창적인 아이콘 디자인 나만의 컬렉션 완성
맵	독창적 맵 디자인 맵별 전략적 다양성

3.2 게임 기획

3.2.1 세계관 및 콘셉트

시놉시스

왕국에 도전장이 왔다!

각지에서 몰려드는 적을 막아내고, 성을 지켜내야 한다. 하지만 절망할 필요는 없다.

기사, 석궁병, 마법사 등 다양한 전우들이 당신의 명령을 기다리고 있다.

당신은 새로운 사령관으로서 골드를 현명하게 사용해 병력을 배치하고,

매 라운드 강해지는 적의 공격을 막아내야 한다.

전투는 치열하지만, 매번 새로운 전략과 조합으로 승리를 만들어가는 것은 당신의 몫이다.

왕국은 아직 희망으로 가득 차 있다.

“얼마나 오래 지켜낼 수 있는가”는 단순히 생존이 아닌, 끝없는 도전과 기록을 세우는 즐거움이다.

이제 성문을 열고, 전략과 지혜로 적을 맞이하라!

콘셉트

구분	콘셉트
플레이어	왕국을 지켜내는 새로운 사령관
골드	골드를 소모해 병력 배치 어떻게 사용할지는 사령관의 지혜에 달려 있다
성	수백 년 동안 왕국을 지켜온 요새로, 병사들과 시민들에게 희망과 용기의 상징
맵	왕국을 둘러싼 대지는 끝없는 전장의 무대다 - 전란의 숲: 생명의 숲이라 불리는 성스러운 땅. 울창한 나무들 사이로 전투의 긴장감이 흐른다. - 황금 사막: 눈부신 태양 아래 펼쳐진 모래벌판. 끝없는 모래 언덕은 발걸음을 시험한다. - 격전의 설원: 하얀 눈보라가 덮인 얼어붙은 강이 군세의 발걸음을 가둔다. 차가운 바람 속에서 전사는 오직 의지만으로 싸운다

장르 적합성

본 프로젝트는 전략 자동 전투 장르를 기반으로 하며, 짧은 시간 내 전략적 재미 추구,

기존 멀티플레이 중심 작품과 차별화된 싱글 플레이 특화, 캐릭터 수집 및 시너지 시스템과 장르적 특성의

적합성을 확보.

기존 오토배틀러 장르의 주요 특징을 계승하여, 장르 이해도가 있는 이용자의 손쉬운 적응과 즐거운

플레이 가능성.

3.2.2 게임 시스템 설계

전투 시스템

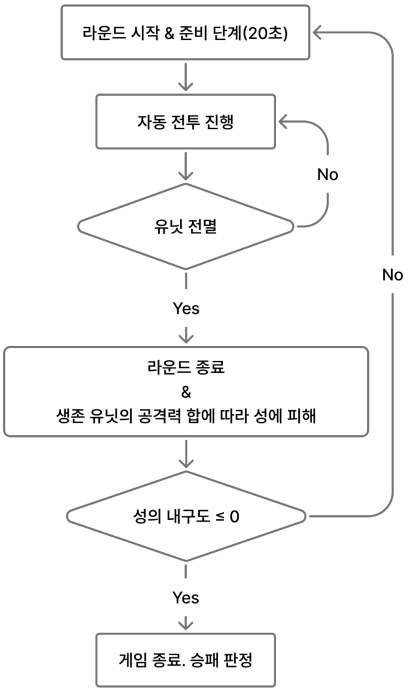
의도

자동 전투의 규칙을 단순화해 라운드 단위 판단과 전략적 배치에 집중

핵심 규칙

- 라운드형 전투 진행, 한쪽 전멸 시 라운드 종료
- 플레이어 패배 시 게임 종료

게임 흐름



요약

매 라운드 적의 공격을 막아내며 최대한 오래 버팀

배치된 유닛들이 자동으로 전투 수행

한쪽 유닛이 모두 전멸하면 라운드 종료

라운드 종료 시 상대의 생존 유닛 수와 공격력에 따라 성의 내구도 감소

성의 내구도가 0이 되면 게임 종료

강화 시스템

의도

수집 목표와 유닛 전력 증강을 제공하여 게임의 중장기 동기를 부여

핵심 규칙

동일 유닛 3기 수집 시 자동 합성

등급 상승 시 체력·공격력 배수 상승, 크기 각 1.5, 2배 적용.

게임 흐름

상점 구매/드롭 → 보드에서 동일 유닛 집계 → 강화 조건 충족 시 즉시 합성 → 스탯 재주입·UI 갱신

UI/조작

상점에 카드 이미지를 클릭하여 유닛 구매

유닛의 크기로 시각화

시너지 시스템

시너지	활성화 조건	보너스	역할
Guard	2+ 유닛	자가 치유 +3HP/s	전선에서 팀을 지키는 방패
Arcane	2+ 유닛	공격력 +4	원거리 화력 지원 강화
Vanguard	2+ 유닛	받는 피해 -20%	전투 최전방에서 피해를 흡수

의도

강력한 유닛 하나에 의존하는 플레이를 줄이고, 편성·배치 단계의 전략성을 강화

핵심 규칙

같은 시너지 태그 유닛 수 ≥ 조건이면 활성화

게임 흐름

준비 단계에 편성 계산 → 전투 시작시 보너스 부여 → 라운드 종료 시 초기화

UI/조작

배치 화면 좌측에 시너지 효과 확인 가능

자원 및 배치 시스템

의도

짧은 라운드 루프에서 빠른 경제 판단과 배치 전술의 재미 제공

핵심 규칙

기능	내용
시작 자원	골드 5, 체력 100, 레벨 1
라운드 종료 보상	기본 5골드 + 보유 골드 10당 1골드 이자 (전투 후 준비 단계 지급)
레벨업	골드 4 사용 시 배치 한도 +1, 최대 9
상점	라운드 종료 시 자동 새로고침, 2골드로 수동 새로고침 가능
유닛 구매	가격만큼 골드 차감 후 인벤토리 스폰
상점 유닛	유닛 5개 무작위 등장, 전장/인벤토리에 자유 배치 가능
배치 제한	현재 배치 한도 기준 배치 한도를 초과한 유닛 배치 시도 시 강제 인벤토리 이동

게임 흐름

수익 정산/상점 갱신 →유닛 구매/배치 → 전투 시작.

UI/조작

게임 화면 하단에 상점 5슬롯, 새로고침 버튼(2골드), 레벨업 버튼(4골드) 표시

맵 시스템과 Centerzone

의도

맵마다 다양한 환경을 조성하여 새로운 전투 경험 제공

맵 종류

- 전란의 숲
- 황금 사막
- 격전의 설원

핵심 규칙

각 맵에 Centerzone 1개 존재. Centerzon은 전투 단계에서만 활성화

Centerzone : 맵마다 존재하는 특수 지형. 맵마다 서로 다른 효과를 주어 전략성을 극대화함.

맵	Centerzone 효과	활용 이펙트
숲	유닛의 이동속도 +100%	가속 효과를 시각적으로 표현하기 위해 상승 파티클 사용
사막	유닛의 이동속도 -50%	감속 효과를 시각적으로 표현하기 위해 하강 파티클 사용
눈	4초마다 유닛 1초 정지	정지 효과를 시각적으로 표현하기 위해 눈이 떨어지는 이펙트 추가

게임 흐름

유닛이 Centerzone 진입 시 테마에 맞는 효과 적용. 영역 이탈/사망/라운드 종료 시 효과 제거

UI/조작

전투 진입 시 Centerzone 영역 시각화 (상승, 하강 파티클, 눈 입자)

보상 시스템

의도

게임 성과를 점수·재화로 환류해 반복 동기 강화

핵심 규칙

보상	내용
처치 보상	적 유닛 1기당 +10점
게임 종료 보상	레벨·골드 지급, 누적 점수는 **랭킹 패널**에서 확인 획득한 골드로 프로필 아이콘 구매 가능

게임 흐름

라운드 종료 시 점수 합산 → 게임 종료 시 누적 보상 지급

UI/조작

로비 화면에서 점수/랭킹/보유 골드 확인 가능

3.3 구현 완성도

3.3.1 핵심 기능 구현

CenterZone

유닛 성능만으로 승부가 나는 반복 패턴을 줄이고, 맵 자체가 경로를 선택하도록 설계한 환경 효과 시스템.

기능	내용
활성 단계	준비 단계 비활성 → 전투 단계에서만 활성 → 라운드 종료 시 즉시 비활성
영역 감지	콜라이더를 isTrigger로 운용해 유닛의 진입/체류/이탈을 이벤트로 처리
모드 통합	하나의 존이 아래 모드 중 하나로 작동 ● Fast: 내부 동안 이동속도 배수 가속 ● Slow: 내부 동안 이동속도 배수 감속 ● IceTrap: 내부에 있는 각 유닛 기준 주기로 스턴 (기본 4초, 개별 쿨다운으로 락 현상 방지)
속도 제어	“속도 컨트롤러”가 NavMeshAgent의 기본 속도를 캐시 영역 내부에서는 기본속도 × 배수로만 수정. 이탈·사망·라운드 종료 시 즉시 원복.
이펙트 연동	영역이 활성화될 때 루프 VFX를 1회 스폰 후 재사용. 콜라이더 사이즈를 읽어 자동 스케일하고, 미세 보정·지면 간섭 방지로 튜닝. 에디터에서 크기 변경 시 즉시 반영.

전란의 숲

황금 사막

격전의 설원

전투 로직

구분	기능
ChampionController	유닛 1기의 전투 전반을 총괄. 전투 중 시너지 적용 실행
ChampionAnimation	애니메이터 파라미터 제어와 공격 종료 신호를 컨트롤러에 전달
AttackBehaviour	애니메이션 상태머신 훅. 공격 상태 종료 타이밍을 이벤트로 알려 피해 판정 타이밍을 보장
SpeedController	NavMeshAgent 기본 속도 캐시/배수 적용으로 가속/감속 처리
CentralZone	가속/감속, 스턴 트리거. 전투 단계에서만 유닛에 효과 부여
GamePlayController	전투 시작/종료 신호, 라운드 전환과 UI 갱신 총괄
Champion	전투에 필요한 원본 데이터 제공(프리팹, 투사체, 스탯, 사거리 등)

유닛 합성

구분	기능
Champion	등급에 따른 스탯 소스. 유닛 업그레이드 시 상위 단계 데이터로 재주입
ChampionController	교체된 유닛의 런타임 상태를 유지하면서 새 스탯을 반영
GamePlayController	상점 구매 성공 시 합성 트리거 발동 → 동일 유닛 수 집계 → 합성 조건 충족 시 유닛 업그레이드 실행
GameData	전체 유닛/타입 카탈로그. 합성 로직이 유닛 정의를 조회할 때 참조.

조합 시너지

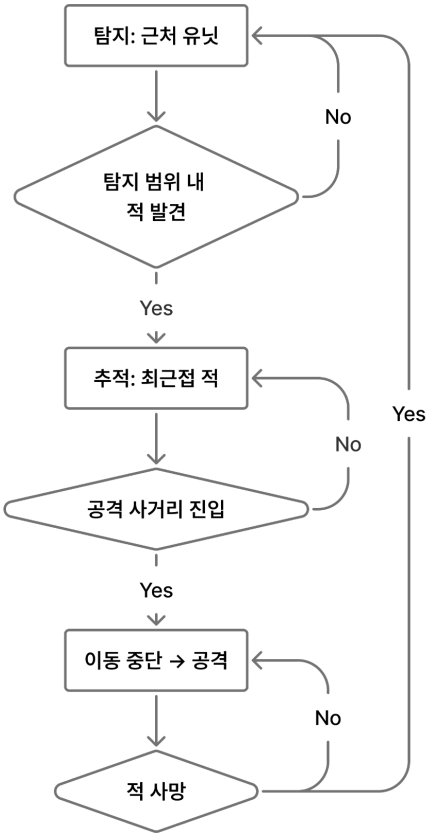
구분	기능
ChampionType	유닛이 속한 타입 정의와 아이콘/이름 표시
ChampionBonus	“동일 타입 N명 이상” 시 발동할 보너스 규칙 정보 저장
ChampionController	공격 판정 시 공격형 보너스 적용 (Damage) 피격 시 방어형 보너스 적용 (Defense)

GamePlayController	보드의 타입별 유닛 수 재집계, 활성화 여부 판정, 활성 보너스 목록 관리 및 UI 갱신.

이동 및 공격 로직

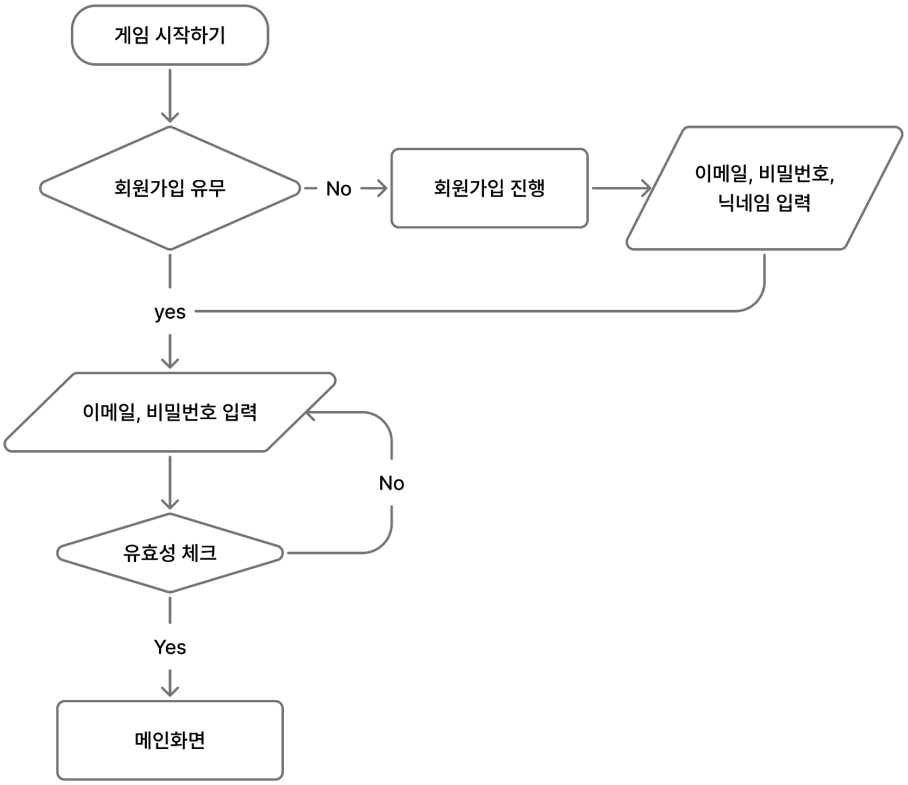
유닛 드래그시 평면 교차점을 따라 위치를 보간

전투 시작 시 NavMeshAgent를 활성화하고 목표를 찾으면 이동

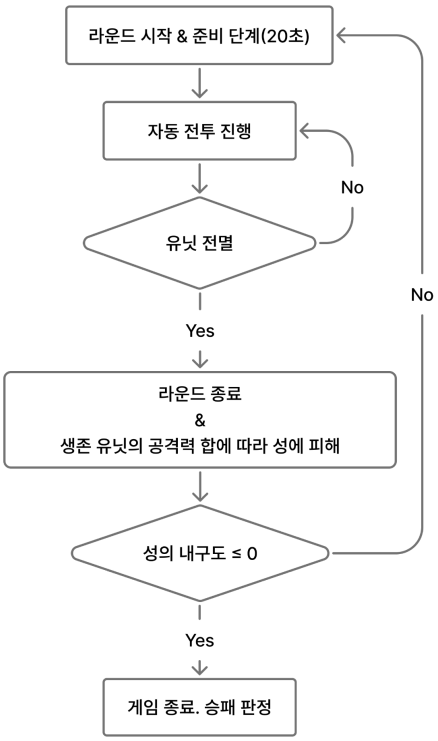


3.3.2 게임 진행 흐름

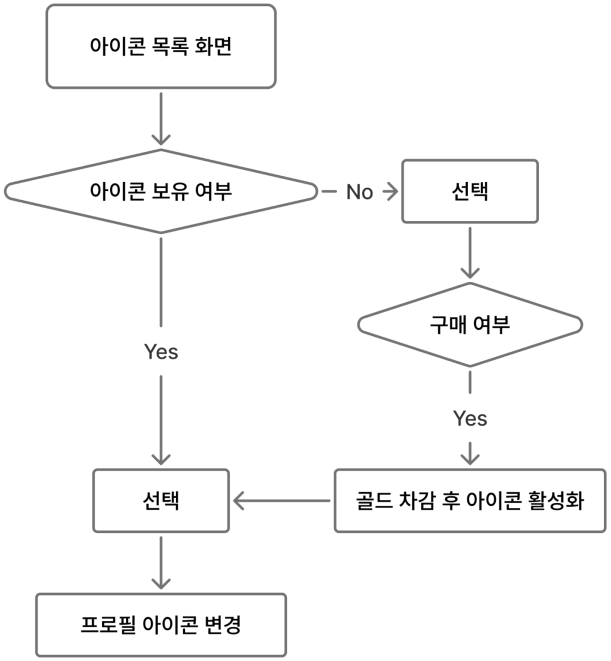
로그인


게임 시작하기
회원가입 유무
No
회원가입 진행
이메일, 비밀번호,
닉네임 입력
yes
이메일, 비밀번호 입력
No
유효성 체크
Yes
메인화면

게임진행


라운드 시작 & 준비 단계(20초)
자동 전투 진행
No
유닛 전멸
No
Yes
라운드 종료
&
생존 유닛의 공격력 합에 따라 성에 피해
성의 내구도 ≤ 0
Yes
게임 종료. 승패 판정

아이콘



3.3.3 기술 안정성 및 버그 최소화

렌더러(Renderer) 변환

Built-in Render Pipeline과 URP 간 호환성 문제 발생

프로젝트 간 리소스 이동 시 머티리얼·셰이더 호환 불가 현상 발생

Unity URP Converter 활용을 통한 머티리얼 자동 변환 적용

게임 내 최적화

구분	내용
이벤트 기반 데미지 계산	프레임 폴링 없이 애니메이션 종료
NavMesh 비용 절감	전투 시작 전까지 에이전트 비활성
재탐색 스로틀링	타겟 상실 시에만 0.5초 주기로 스캔, 불필요한 거리 계산 감소
이펙트 재사용	동일 프리팹 이펙트는 지속시간만 갱신에 중복 컴포넌트
Scriptable Object	데이터 중복 최소화와 런타임 부하 감소로 인한 메모리 및 성능 최적화
오브젝트 풀링	재사용을 통한 생성·소멸 비용 절감으로 메모리 성능 최적화

3.3.4 게임 밸런스

캐릭터 디자인

유닛	코스트	타입	체력	대미지	사거리
Archer	1	Ranger	65	14	15
Knight	2	Vanguard	130	19	2
Mage	3	Ranger	75	28	10
Mount Knight	4	Vanguard	160	23	3
Crossbowman	2	Ranger	80	20	15
Heavy Infantry	3	Guard	170	17	2
Spearman	2	Guard	95	16	3
Swordman	1	Guard	105	13	2

AI 디자인

시작 시: 맵 준비/재시작마다 무작위 유닛 1기 지급

라운드 종료 후: 진영 리셋 + 빈 슬롯이 있으면 무작위 유닛 1기 추가

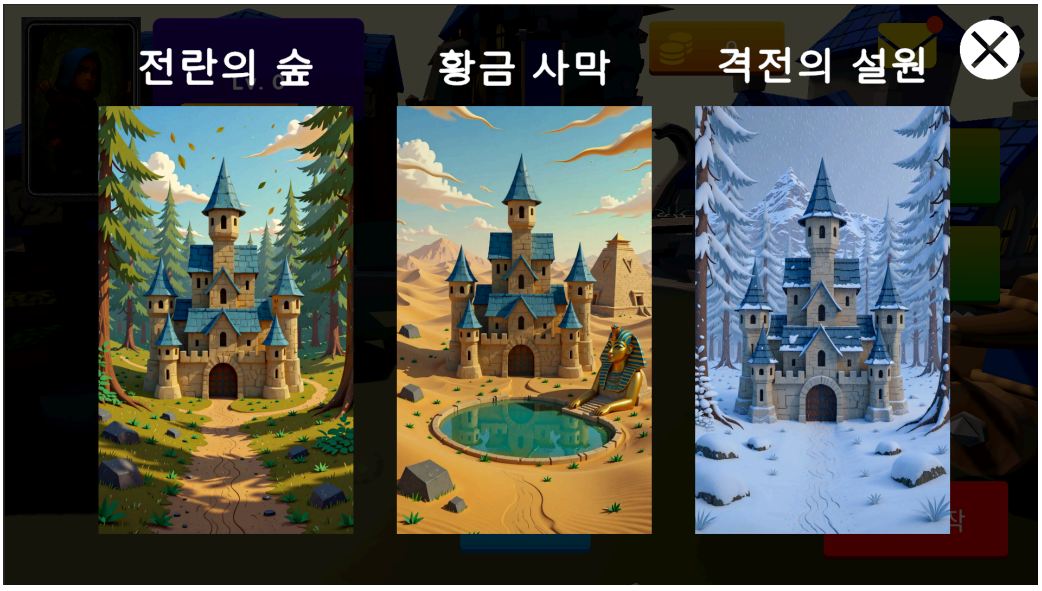
강화: 동일 유닛 3개 → 2성, 2성 2개 → 3성 (플레이어와 동일)

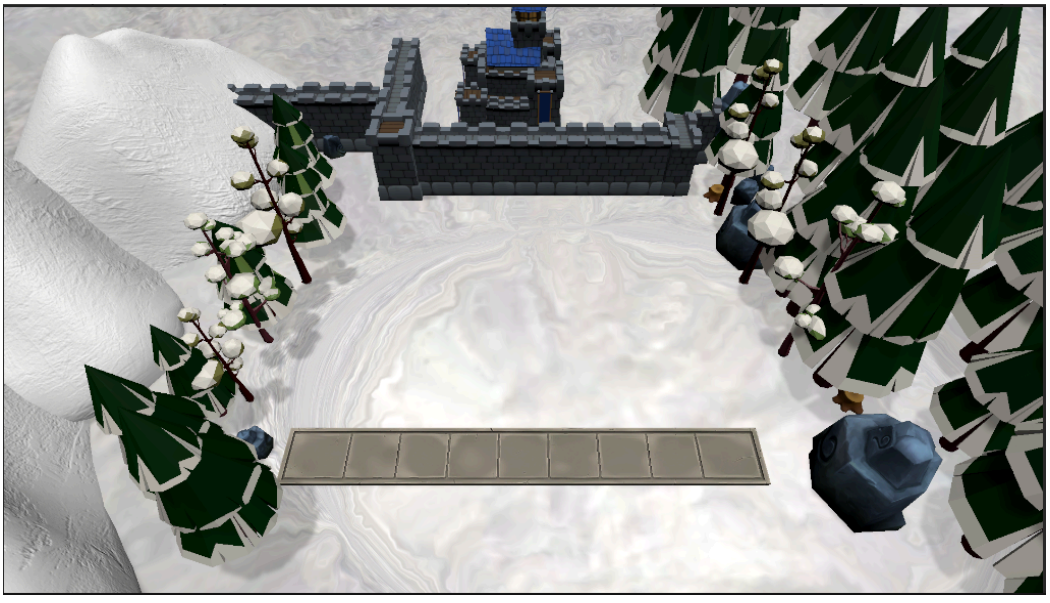
성능: 플레이어와 동일 프리팹·스탯 적용, 등급별 능력치·크기 배수 동일 반영

3.4 아트·UX/UI 구성

3.4.1 유닛

3.4.2 맵





3.4.3 인게임 UI

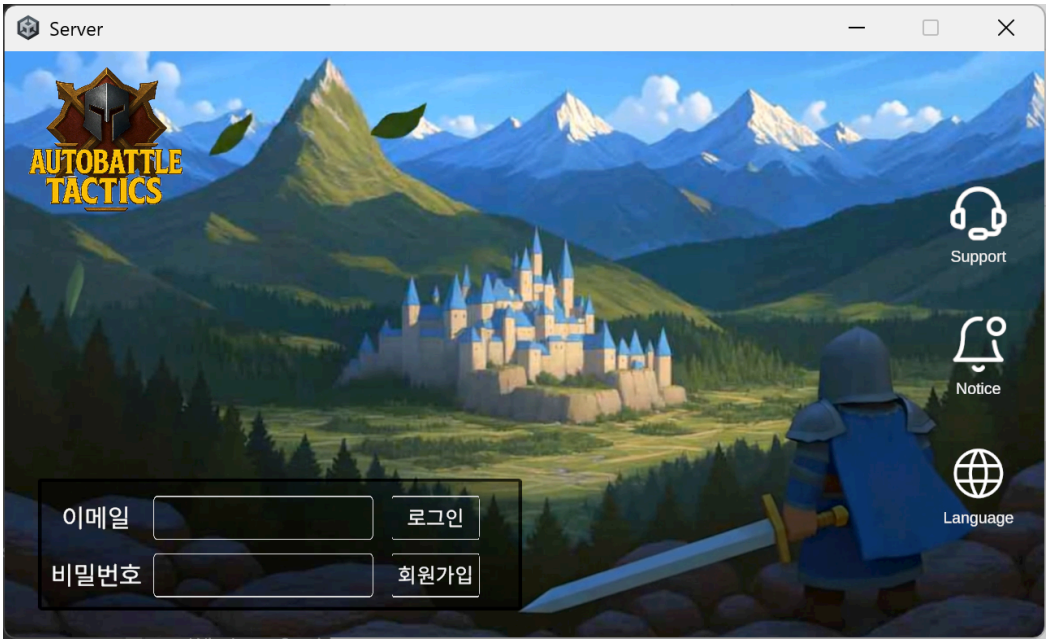
남은 체력 / 배치가능 유닛 수 / 골드

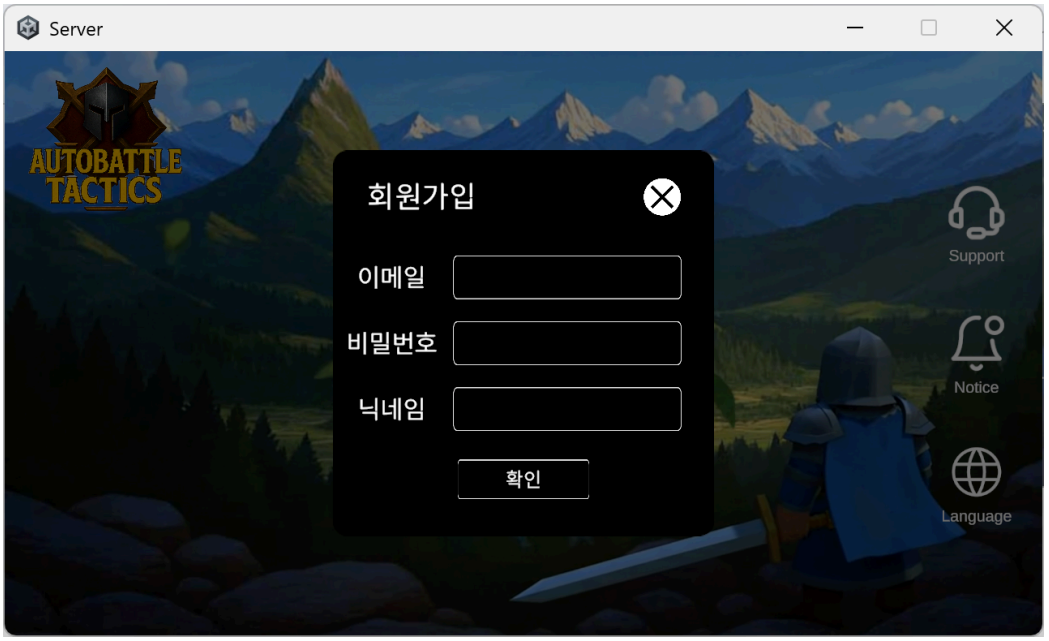
유닛 배치 시 시너지별 유닛 수 표시

유닛 카드 - 이미지와 시너지, 골드, 이름 표시

3.4.4 게임 플레이 화면

로그인 / 회원가입 씬





메인 씬

메인 씬 - 아이콘 판넬

보유중인 아이콘 확인/변경 가능

메인 씬 - 랭킹 판넬

qqqq
21000
랭킹
qqqq
6
순위
80
점수
20
경기수
1 123 2830
2 123@qq.com 120
3 mnmn 100
4 gtgt 100
상점
랭킹
캐릭터 변경
게임 시작

게임 씬

1
4 / 5
13
Ranger 1 / 2
Guard 1 / 2
Vanguard 2 / 2
Reroll
2
LevelUp
4
HeavyInfantry
Swordman
Swordman
Knight
Crossbowman

3.4.4 AI 이미지 생성 과정

개요 및 워크플로우

확산(Diffusion) 모델과 저랭크 적응(LoRA) 기술을 활용한 생성형AI 워크플로우 구축을 통해

인게임 삽화 이미지를 생성형 AI로 제작

기술 스택	구성 요소	도입 이유
기반 모델	SDXL (Stable Diffusion 오픈소스 모델)	오픈소스 환경에서 자유로운 제작 가능
학습 프레임워크	Kohya_ss (LoRA 학습 프레임워크)	LoRA 훈련에 특화된 프레임워크로, 효율적인 학습을 지원
데이터 처리	Rembg, taggui, unity C# 스크립트	AI 비전 기술을 활용해 배경화면 제거 전처리 작업을 자동화
개발 환경	Unity, Python	Unity는 데이터 수집을 위한 환경으로, Python은 이미지 학습 및 생성 프레임워크 구동을 위해 사용
추가 도구	ControlNet (Openpose,IP-adapter)	생성물의 구도와 포즈를 정밀하게 제어하여 원하는 장면의 이미지를 생성

1. 데이터 수집 단계

- 모델링 캡쳐를 위한 Unity 씬 구성 (1:1 화면 종횡비, 회색 배경화면)
- Unity 자동 캡처 스크립트 작성 (화면 비율 조정 및 전면, 후면, 측면 등 다양한 각도)
- 초기 데이터셋 수집 (16개 캐릭터 모델링에 대해 10장씩 160장의 데이터셋 수집)

2. 데이터 전처리 단계

- 객체 탐지 기반 배경 제거 도구(rembg)
- 비전 트랜스포머 기반 라벨링 도구(taggui) + 기반 모델 병합을 위한 공용 태그 수동 라벨링

3. 데이터 필터링 및 증강

- 초기 데이터셋에서 표현력이 부족한 각도를 제외하여 이미지 80장 선별
- flip을 이용한 좌우반전 데이터 증강

4. LoRA 모델 학습

- 모델 표현력과 관련된 하이퍼파라미터 최적화 (Network Dimension와 Network Alpha)
- 학습 속도및 메모리 사용량과 관련된 하이퍼파라미터 최적화
  (8비트 AdamW 옵티마이저 및 배치사이즈 2배)

5. AI 이미지 생성

- LoRA 모델 적용
- ControlNet으로 포즈 제어
- 일관된 스타일의 캐릭터 이미지 생성

6. 후처리 및 Unity 통합

- 생성 이미지 후처리 및 인게임 적용
- I2V 생성형 비디오 모델을 이용한 애니메이션 확장

AI 모델 설계

LoRA(Low-Rank Adaptation)

LoRA는 대형 AI 모델을 효율적으로 세밀하게 조정하는 기술(Fine-Tuining)

전체 모델을 재학습하는 대신, 저차원 행렬을 통해 특정 지식을 추가하는 방식으로 작동함

- **핵심 작동 원리**
  기존 모델 가중치 W + LoRA 저랭크 행렬 (A/B) = 새로운 가중치 (이미지 생성 결과 변형)
  - B: 다운 프로젝션 행렬 (d x r)
  - A: 업 프로젝션 행렬 (r x k)
  - r: rank (저차원 표현)
  - r의 저차원을 가진 행렬만 학습시키므로 모델 전체 가중치 수정 필요 X,
    메모리 최적화 및 학습 속도에서 강점
- **주요 특징**
  - GPU 메모리 효율성 (저사양 GPU에서도 학습 가능)
  - 빠른 학습 속도 (전체 가중치 갱신 기준 수개월 → 저랭크 행렬 갱신 기준 수시간)
  - 플러그 앤 플레이 방식 (모델과 병렬로 결합되므로 기존 모델 손상 없음)
  - 비용 절감

데이터 수집

- **Unity 씬 자동 캡쳐 스크립트**

  학습 데이터로서 Toony Tiny RTS Set 유니티 에셋 번들을 사용
  Unity 환경에서 자체 개발한 자동 캡쳐 시스템을 통해 수집

  - **캡처 해상도**: 1024px*1024px 해상도의 이미지.
    -> SDXL 원본 학습 데이터셋에 맞춰 해상도 정규화. 잠재공간 왜곡 방지.
  - **배경 설정**: 배경 제거 작업에서 Segmentation의 용이함을 위해 단색 회색 배경으로 설정.
  - **초기 수집 결과**: 160장의 데이터셋 확보

데이터 전처리 및 라벨링

- **이미지 배경 제거**: 데이터셋의 배경을 제거하여 이미지에서 캐릭터 Feature만 학습
  - **입력**: 단색 회색 배경 이미지 160장
  - **처리 도구**: github.com/danielgatis/rembg
  - **처리 시간**: 약 40분 (수작업 없이 백그라운드에서 처리)
  - **출력**: 투명 배경 PNG 파일

- **데이터 라벨링**: 텍스트 프롬프트의 Conditioning 설정을 위한 태그 기반 라벨링

  - **공통 태그 설정**: '1boy', 'solo', 'chibi', 'medieval', 등 데이터셋의 보편적인 특징을 나타내는 태그를 설정. 기반 모델의 지식과 병합되어 고품질의 이미지 생성 유도
  - **작업 시간**: 약 90분

품질 확보 전략

-> 초기 프로토타입의 출력 : 200스텝 단위의 이미지 변화

- 초기 프로토타입에서 발견된 문제점
  - 800스텝 프로토타입 LoRA 모델에서 LoRA의 효과가 불분명함
  - 출력 이미지 캐릭터의 등 뒤에 아티팩트 출현(궁수를 출력했으나 창이 동시에 표현)
  - 현재 데이터셋 그대로 스텝을 늘리면 특징 학습보다 아티팩트 문제가 더 부각

1. 전처리 최적화

1.1 데이터 선별:
초기 데이터셋에서 측면 및 후면 이미지의 특징 표현 불분명함(얼굴 표현 부족)
일부 각도에서 발생하는 모델링 아티팩트(예 : 시야각 문제로 무기같은 오브젝트가 과장)
-> 표현력이 부족한 측면, 후면과 아티팩트가 식별된 이미지를 데이터셋에서 제외,
160장 -> 80장으로 데이터셋 선별

1.2 데이터 증강: Flip을 통한 데이터 증강
선별된 80장의 데이터는 리핏(Repeat)을 2배로 적용하는 대신 좌우반전(Flip) 증강을 적용해 절반으로 줄어든 데이터셋 규모를 보완

1.3 트리거 워드 설정: 기반 모델에 없는 고유 태그인 'TT_RTS'를 트리거 워드로 설정
해당 태그를 가진 데이터셋만을 단독으로 호출 가능
-> 기반 모델의 지식 반영이 지나치게 강해 LoRA의 표현력이 떨어지는 문제 개선

최종 하이퍼파라미터 최적화 과정

- **초기 프로토타입**

파라미터	설정값	결과
Steps	800	프로토타이핑을 위한 낮은 스텝수 설정
Batch Size	1	기본값 사용
Network Dim	32/32	기본값 사용
학습 시간	1시간 30분	짧은 학습 시간

⇒ **결과**: LoRA의 표현력이 부족하고, 아티팩트 등 과적합 징후가 발견

- **최종 최적화 파라미터**

파라미터	최적화 값	최적화 근거
Batch Size	2	AdamW8bit 사용으로 생긴 VRAM 여유에 따라 2스텝씩 묶어서 처리
Epoch	5	적정 학습량인 2000스텝 고려
Steps	2000	권장 최소 스텝 충족
Network Dim/Alpha	64/64	벡터 차원을 확장시켜 강력한 LoRA 학습 유도
Optimizer	AdamW8bit	상태변수를 8비트로 처리해 학습 VRAM 사용량 크게 감소
Mixed Precision	bf16	그라디언트 학습 과정 중 가중치 손실 방지
학습 시간	4시간	과하지 않은 학습 시간 소요

⇒ **결과**: 초기 프로토타입과 비교한 결과 과적합 없이 캐릭터의 특징을 잘 학습한 모습

ControlNet 모델 - dw_openpose_full

dw_openpose_full : 관절 인식 비전 모델,

포즈 이미지에 대해서 관절 키포인트를 추정, controlnet을 통해 추정치를 모델에 가중치로 입력

LoRA와 결합하여 캐릭터 모델링 스타일을 유지하면서 다양한 포즈의 이미지 생성 가능

1. 포즈 추출

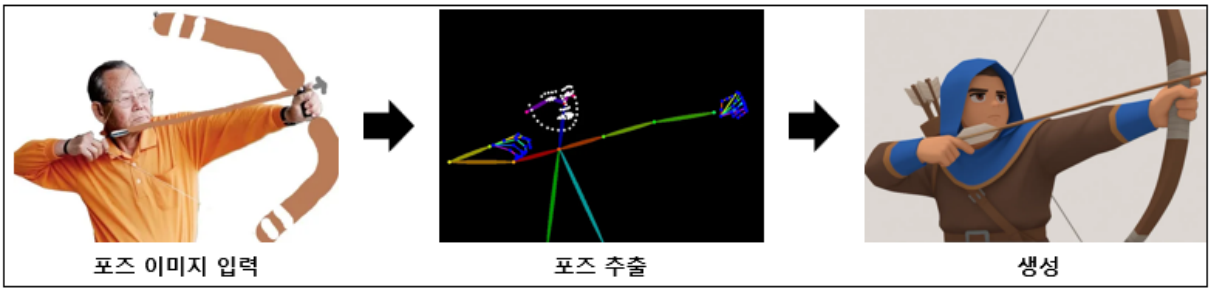
- dw_openpose_full를 통해 입력 이미지에서 전신 키포인트(관절 좌표)를 감지 및 추출

2. 포즈 기반 이미지 생성

- 추출한 전신 키포인트를 ControlNet을 통해 모델에 가중치로 입력
- 캐릭터 LoRA + 텍스트 프롬프트 + 포즈 이미지 → 특정한 포즈의 캐릭터 이미지 생성

3. 일관성 유지

- 생성 시드(seed)와 LoRA, 텍스트 프롬프트를 고정한 상태로 포즈 이미지만 변경
- 포즈와 관련된 가중치만 수정 → 동일한 캐릭터의 다른 포즈 제작 가능



ControlNet 모델 - IP-Adapter

1. 멀티모달 이미지 프롬프팅

- IP-Adapter를 통해 이미지를 입력하면 이미지 프롬프트로서 모델에 가중치에 입력

2. 컨셉아트를 통한 이미지 프롬프팅

- 묘사하고 싶은 컨셉아트(장면 등)를 IP-Adapter를 통해 모델에 가중치로 입력
- 이미지의 특정 Feature만 가중치로 입력될 수 있게 Weight Block 파라미터를 수정
- 입력 블록에 집중하면 Composition, 출력 블록에 집중하면 Style을 강조
- 컨셉아트의 Composition를 집중적으로 활용하기 위해
  출력블록 가중치를 낮추고 입력블록 가중치를 높이면 LoRA 스타일을 유지하며 장면 모사

IP-Adapter에 컨셉 아트 이미지 입력

입력 이미지와 유사한 Composition을 가지는 이미지 생성
IP-Adapter 출력블록의 가중치를 낮게 설정하여
이미지의 Style은 LoRA의 영향을 더 크게 받음

- **온라인 생성 모델 결합을 통한 결과물 보정**

  GPT-Image-1, Google Nano-Banana와 같은 온라인 생성 모델을 통한 후보정. 퀄리티 향상

결과 해석 및 한계점

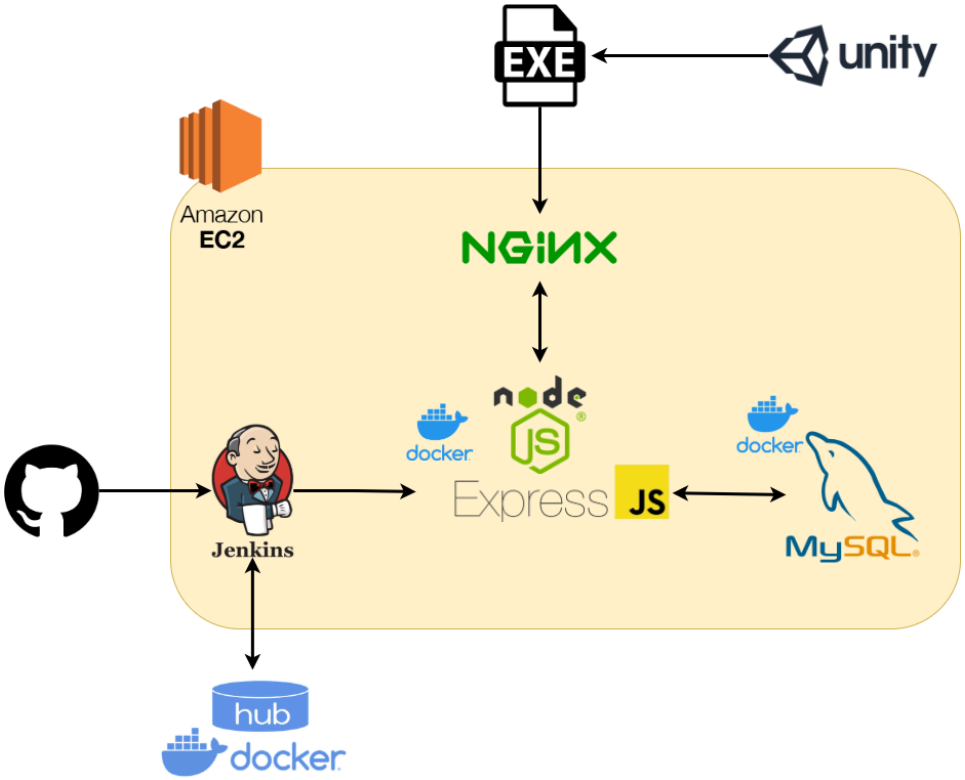
- **성공 요인**
  1. **효율적인 데이터 수집 자동화:** Unity 자동화 스크립트를 통한 효율적인 데이터셋 수집
  2. **최적화된 전처리 파이프라인:** AI 기반 자동화로 라벨링 전처리 소요 시간 대폭 단축
  3. **하이퍼파라미터 최적화**: 저사양 하드웨어에서도 학습 가능한 8비트 옵티마이저 설정

- **기술적 한계**
  1. **데이터셋 규모 :** 80장의 데이터셋은 한 프로젝트의 모든 캐릭터의 모델링(동물 등 비인간형 모델링 등)을 표현하기는 한계가 존재함
  2. **기반 모델 한계 :** SDXL의 CLIP 텍스트 인코더 성능 한계로 세부적인 결과물 조작의 어려움
  3. **스타일 범위:** LoRA가 TT_RTS의 중세 스타일에 특화되어 있어 현대 모델링 등 범용성 제한

- **개선사항**
  - 1. **기반 모델 변경:** FLUX, Qwen-Image-Edit 등 성능이 더 뛰어난 오픈소스 모델 사용
  - 2. **데이터셋 추가 수집 :** Toony Tiny RTS Set 에셋 제작자의 현대 캐릭터 에셋을 포함한 다른 에셋도 활용하여 데이터셋 규모와 지식 범위 확장
  - 3. **프롬프트 엔지니어링 :** 기반 모델의 성능을 더 적극적으로 활용하기 위한 태그 작성
    캐릭터 별로 트리거워드 분리 및 호출(궁수 : TT_Bowman / 검사 : TT_Swordman 등)
    중세(medieval)를 포함해 중세의상(surcoat), 왕관(crown), 투구(visor armor) 등
    캐릭터를 자세히 묘사할 수 있는 태그를 활용

3.5 기술 및 구조

3.5.1 시스템 아키텍처



클라이언트: Unity 6 기반 Windows 실행 파일(Exe) 빌드

서버 구조: Node.js(Express) 기반 REST API 서버, MVC 아키텍처 적용

데이터베이스: MySQL

배포 환경: Docker 기반 컨테이너, AWS EC2 인스턴스 배포

CI/CD 파이프라인: GitHub → Jenkins → Docker Hub → AWS EC2 자동화 구축

3.5.2 주요 모듈 및 서비스 흐름

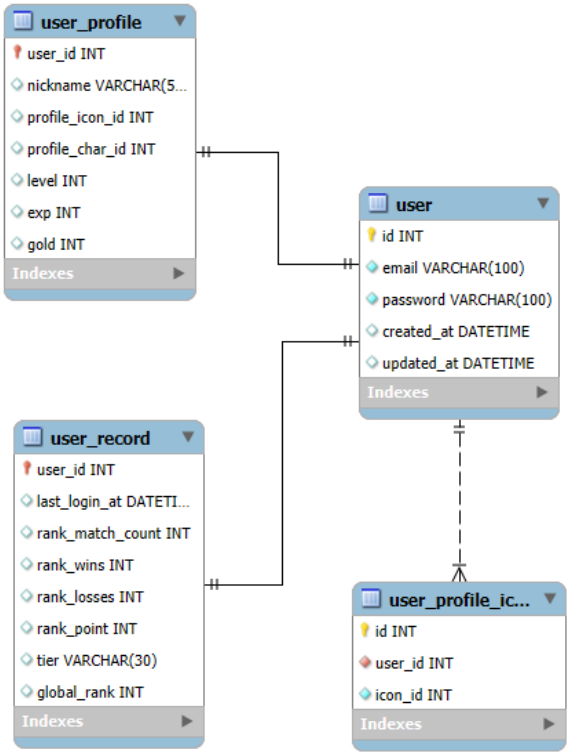
로그인/유저 조회

AuthController → JWT 발급, SessionManager 저장 → API 요청 시 서버 검증 후 데이터 조회 → UI 반영

API 명세서

Swagger 문서 : https://jamsik.p-e.kr/swagger-ui/

ERD



3.5.3 기술 스택 및 도입 목적

Node.js + MySQL: 유저/랭킹 관리 REST API, 데이터 신뢰성 확보

Docker + Docker Hub: 재배포/확장 용이

Jenkins: 빌드/배포 자동화

AWS EC2: 서버 운영 및 인프라 제어

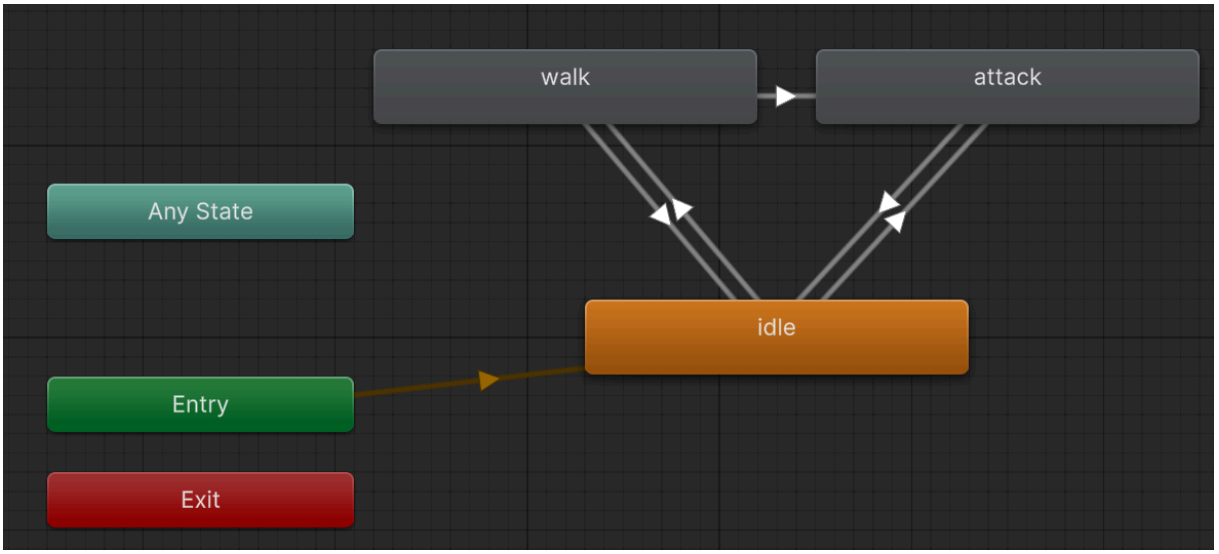
NGINX: Unity WebGL HTTPS 지원, 인증서 자동 관리

JWT 기반 인증: 무상태, 로그인 성공 시 JWT 발급, 만료 1일

서버 Public IP: 13.61.174.133

Domain: https://jamsik.p-e.kr

3.5.4 애니메이션 적용



우리 게임의 유닛은 Idle, Walk, Attack 세 가지 상태를 가짐. 각 상태에 맞는 애니메이션을 정의하여 유닛의 행동을 시각적으로 표현

애니메이션 컨트롤러(Animator Controller)를 활용하여 상태와 전환을 관리

상태 간 전환(Transition)을 시각적으로 구성하여 유닛의 동작 변화에 따른 애니메이션 전환 구현

위 구조는 소프트웨어 공학의 상태 패턴(State Pattern)과 유사한 개념

상태 패턴은 객체의 상태 변화에 따라 동작을 다르게 정의하는 설계 방식이며, 상태 변화(Idle → Walk → Attack)에 따라 서로 다른 애니메이션을 실행하는 구조

이를 통한 코드 가독성과 유지보수성 향상, 새로운 상태 추가 시 확장 용이성 확보